

SIMATSENGINEERING(SAVEETHASCHOOLOFENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3-Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 - Articulation (BL3)	Assessment:	Assessment Tool 3-Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	VAISHNAV	Reg. No.:	192524251
Section/Batch:	2025	Date:	26/08/2026

Code of ConductI VAISHNAV (Name/RegNo)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VAISHNAV

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answers using comparative analysis, it should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section/Topic	Focus	QNos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organizational issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature:	Signature:
Date:	Date:	Date:

COM - AI 3 - COMPARATIVE ANALYSIS

Q1. compare linear Probing and separate chaining for collision handling.

Parameter	linear Probing	separate chaining
clustering Effect	Suffers from primary clustering where consecutive occupied slots form clusters.	No clustering problem because collided elements are stored.
Memory usage	Requires only the hash table.	Requires extra memory
Performance under High load Factor	Performance degrades significantly.	performs better even with high load factor.
Large - Scale applications	suitable when memory is limited.	More - Suitable for large - scale app.

Example :-

hash Table size = 10

Keys : 15, 25, 35

linear Probing :-

$h(15) = 5 \rightarrow$ store at 5

$h(25) = 5 \rightarrow$ store at 6

$h(35) = 5 \rightarrow$ store at 7

separate chaining :-

Index 5 $\rightarrow$ 15 $\rightarrow$ 25 $\rightarrow$ 35

Q2. compare quick sort and Merge sort for a large linked list of 10 Million Nodes.

parameter	quick sort	Merge sort
Memory Requirement	In-place, uses less memory	Requires extra space for arrays
cache Behavior	Excellent cache locality in arrays	less cache-friendly for arrays
Time complexity	average: $O(n \log n)$ worst: $O(n^2)$	Best, average, worst: $O(n \log n)$
linked list suitability	Partitioning is difficult in linked lists.	Splitting and merging are easy
Stability	Not stable.	Stable sorting algorithm.

Conclusion :-

* linked lists $\rightarrow$ Merge sort

* arrays $\rightarrow$ Quick sort.

Q3. compare quadratic probing and Double Hashing.

Parameter	Quadratic Probing	Double Hashing
clustering issues	eliminates primary clustering but suffers from secondary.	^{no} Eliminates both primary and secondary.
collision Resolution efficiency	moderate efficiency.	High efficiency.
Dense Hash Tables	performance decreases, when table becomes dense.	Performs better in dense table.
Hash Functions	uses one hash function.	uses two hash functions.

Formulas :-

Quadratic Probing :-

$$h(k, i) = (h(k) + i^2) \bmod m$$

Double Hashing :-

$$h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$$

Q4. compare Best - case and worst - case time complexity of sorting algorithms.

Algorithm	Best case	worst case
Bubble Sort	$O(n)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$

Practical Significance :-

- * Best case shows the minimum execution time.
- * Worst case shows the maximum execution time.
- * Real - world systems often consider worst - case performance for reliability.

Q5. compare Recursive and iterative implementations of quick sort :-

Parameter	Recursive Quick Sort	Iterative Quick Sort
Space complexity	uses recursion stack $O(\log n)$	uses explicit stack $O(\log n)$
Ease of Implementation	simpler and easier to understand.	More complex due to manual stack management.
Risk of stack overflow	High for large inputs or worst case $O(n)$.	low because recursion is avoided.
Performance	slightly slower due to recursive function calls.	slightly faster in some systems.

Conclusion :-

- * Recursive Quick sort is easier to implement
- * Iterative Quick sort avoids stack and overflow.